Chapter 1

INTRODUCTION

1.1: Introduction:

1.1.1: Brief Overview:

Now a days, people are suffering from many unknown diseases and their ratio is increasing day by day so does the number of ill patients. It seems to be a very tough job for doctors to maintain and keep record of their patients in long rough papers or just memorize the record and history of the patient and record of the medicines.

1.1.2: Patient In Pocket:

"**Patient in Pocket**" is a doctor kit to keep record of their medicines, patients and companies in single app. It is a convenient app that helps doctors to keep their records in their Smartphone.

"Patient in Pocket" is an assistance app for Android and IOS. It is an Android base Smartphone app that enables doctors and physicians to work free from paper or documentation. This app provides user to maintain their clinic record in their Smartphone.

It have the following core functionalities:

- This application can maintain patient personal record, health record, health history, checkups and checkup history.
- This application can maintain Medicines records (Strength, Name, Associated Company, Type etc).
- Through this app, User can maintain Companies records and information.
- You can check all of your records, insert new ones, update the existing, delete unwanted record.
- This application is very User Friendly and working very fast.
- This application is very fast any one can use without difficulties.

1.1.3: Scope:

The Patient In Pocket will perform its operation on Android Platform 1.1 and up. The system first need to be installing on Android Smart Phone or Tablet from Android Market. There is no need of special software or hardware of its installation or working. Internet Connection is not

necessary for its working. Once installed, the system will up and running and user can perform its functionalities. Some of the benefits are enlisted below:

- **Improved Efficiency**

This app keeps nurses and doctors from literally needing to shuffle through file cabinets and loose papers every time they need to access a piece of information about a patient. With Patient In Pocket, there's no need for paper work, lets this smartphone software be a part of a patient's file instead of being stored on a remote computer in a different building or department.

- **Reduced Costs**

Time is money, and reducing wasted time looking through medical records leads to increased productivity. When you do not need to send paper statements to patients this facility can save money. You also save time when patient records are inserted, updated in instance.

- **Potential for Increased Collaboration**

Physicians and doctors are busy. They have many patients to see, and it can be difficult to schedule in-person meetings with patients to discuss the patient's situation. If this software is available on workstations and other devices, such as smartphones and iPads, physicians and doctors can more easily schedule remote meetings to plan the patient's care.

- **Benefits for Patients**

Patient in Pocket, medical record software is more than just a convenience to doctors and other providers. Patients don't need to keep the record every time they went to visit doctor. It create an healthy and satisfactory atmosphere between doctor and patient.

1.2 Introducing Patient in Pocket system:

1.2.1 Problem Statement:

The problem with the existing system is inaccuracy, unreliability, inefficient use of time, money and human resources in visiting attractive places.

1.2.2 Current System:

Currently, most of the translation and visiting in the world are handled and managed manually, using human brain, paper and pencil approach, without the involvement of computer and technology. This manual approach requires a lot of human resources, papers and pencils,

time, efforts, files and cupboards to store this data and use the data later. This approach has a lot of drawbacks, some of which are briefly mentioned in the following headings.

1.2.3 Drawbacks of the Current System:

The first loophole of this approach is difficulty and inaccuracy in learning and storing its words. In the current era, data has become the most valuable asset. Using this approach, data is kept in human memory with no proper management and learning of that much number of languages is impossible. Secondly, human brain is not, much efficient to learn and differentiate number of languages of the world. Also, the amount of money and time needed for this approach cannot be left unmentioned. The process is very tedious and time consuming. Keeping all these factors in mind, this system has to be replaced by some newer, accurate and efficient system, which is not possible without the involvement of computers and software solutions.

Also people face difficulties to get the correct location and have too much energy wastage and time consuming.

1.2.4 Working of System Proposal:

My main goal was to develop an intelligent software that will help the user to

- Keep patient personal record
- Provide patient checkups, checkup history
- Provide patient findings, findings history
- Provide accuracy and efficiency
- Have user interface for user to interact differentially
- Give automatic response to the user queries
- Little or no user interaction with the software
- Show the details of the each record
- Automatic and accurate output
- Use Technology related solution

- Work according to the latest requirements
- Easy to Use
- Less or no human brain usage.
- Less human resources required.
- Less time required.
- Less amount of money required.
- Less effort required.
- SMS receiving about record
- Medicine record facility
- Company record facility

1.2.4 Tools & Technologies:

This system is developed using

- *Android Studio/Eclipse*
- *Microsoft Word*
- *Paint, Adobe Photoshop*
- *Genymotion emulator*
- *Browser*
- *Smartphone (Samsung J2)*

1.3 Feasibility study

An estimate is of whether the identified user needs may be satisfied using current software and hardware technologies. This study decides whether the proposed system will be effective from business point of view and if it can be developed under given budgetary constraints.

It's the test of system proposal according to its workability, impact of the organization/user, ability to meet needs and effective use of resources.

It may include:

1. Economic feasibility

2. Technical feasibility

3. Operational feasibility

4. Legal and Ethical feasibility

5. Schedule feasibility

1.3.1 Economic feasibility:

Economic feasibility means whether the cost of use of any software in the development process, implementation of the system maintenance of the system and technical training of the system is economical to the customer or not. Economically this system is feasible as my organization as well as the customer has all the resources required to develop the system.

1.3.2 Technical feasibility:

Here, I study whether the technology needed for the system development is available. It also describes whether the existing system can be upgraded to use new technology and whether the organization has expertise to use it.

Building Operating system	Win XP Pro, Win 7, Win 8, Win 8.1
Testing Operating System	Android 1.1 and up.
Implementation tool	Eclipse, Android Studio
Testing tool	GenyMotion emulator, Android Smart Phone
Documentation tool	MS Word
Document tools	Paint, photoscape

1.3.3 Operational feasibility:

The system to be developed will satisfy all the objectives, covering all the scopes mentioned above maintaining accuracy, performance, efficiency and privacy.

1.3.4 Legal and Ethical feasibility:

The system is free of any infringements or liabilities. It’s not violating any legal or ethical values.

1.3.5 Schedule feasibility:

Time evaluation is the most important consideration in the development of the project.

Chapter 2

SYSTEM ANALYSIS

2.1 System Analysis

Analysis is the main part of project development. Most of the time spent in project development is given to analysis. System analysis was done using prototyping and object oriented methodology.

2.2 Analysis

Good analysis helps in making perfect software. There are two main parts of the analysis.

- **Requirement Analysis**
- **Domain Analysis**

2.2.1 Requirement Analysis

The purpose of analysis is to provide a model of the system's behavior. In conducting the project, Object Oriented approach is adopted. Object oriented analysis is a method of analysis that examines the requirements from the perspective of the classes and objects found in the vocabulary of the problem domain. In requirement analysis we define use cases diagram containing use cases, actors. A first step in analysis is to extract scenarios, or *use cases* that describe the behavior of a system from an external user's perspective.

System Requirements

The requirements of the system are the most important document in the analysis phase. Requirements of the system define what the system should do, what functionality the system should meet when it is completed and what constraints it has to follow. The requirements of the system should be known before starting to develop the system. Two types of the requirements are defined for description of functionality of Hide and Seek System.

- Functional Requirements
- Non-Functional Requirements

Functional Requirements

These are the statements of services the system should provide, how the system should react to particular inputs and how the system should behave in particular situations. The Functional requirements

define the functionality of the system. It is necessary to define what functionality the system has to achieve. The functional requirements of the system are defined as:

GUI of the application

- The GUI of the application should provide all the necessary options like insert record, delete record, update record and view record. which are necessary for this type of application in the form of buttons and dropdown menus so that user of the application can select an option simply by long clicking buttons and dropdown menu options.

- The buttons should be maximum possible user friendly so that the user could understand the meaning of the buttons.

- All the buttons should display the toast and also the related information right after every execution of the function.

Non-Functional Requirements

Non-Functional requirements define the constraints on the system related to development and performance of the system. The following are the non-functional requirements of the system

- **Portability**

The application should run on any version of Operating System including Jellybeans, lollipop, Kit Kat, Marsh Mallow etc. The application will run on the systems every Android based Smart Phone e.g. qmobile ,voice , Samsung etc.

- **Documentation and source code**

The full documentation of Analysis and design has to be delivered to user. Source code has to be delivered with software to the company.

- **Easy to use**

The interface will be user friendly enough that even new users of android application fell no problem while using the application

- **Tool and Language**

Java is high level, pure Object Oriented Language. It allows developer to collect process and utilize data to create a desired output. It has J2ME version and have libraries for development of apps for Android Operating System.

Problem Statement:

The system we want to develop is General purpose project that will provide the facility to the user to set patient personal record as well as checkup record, also the findings record of patient. It also provides clinic the facility to set the record of their medicines and company from which they import/export medicines. The user may be any person of use android smart phone or tablet. Share the record and send SMS.

2.2.2 Domain Analysis:

Conceptual domain analysis yields common ground for each specific analysis. For domain analysis we have used **RUP** (Rational Unified Process). The Iterative Software Process works well with Object Oriented methodologies. It helps identify and address risks early in the process leading to more robust and better quality systems. Here is brief description of the process.

This Iterative, risk driven process is divided into four primary phases:

Inception

Establish the business rationale for the project and decide on the scope of project. Sponsor agrees to no more than a serious look at the project.

Elaboration

Collection of detailed requirements, high-level analysis and design to establish baseline architecture, and create the plan for construction. Elaboration takes about 1/5 of the total length of the project.

1. Try to discover all the potential use cases.
2. Schedule interviews with users for the purpose of gathering use cases.
3. Identify the risks in the project (requirements risks, technology risks, skills risks, political risks).
4. Build baseline architecture.
5. Planning – Categorize use cases (high priority, architectural risk, schedule risk). Perform iterations on the use case.

6. Finished when (1) developers can feel comfortable providing estimates, to the nearest person-week of effort it will take to build each use case, (2) significant risks have been identified, and the major ones are understood to the extent you know how to deal with them.

Construction

Builds the System in a series of iterations (mini-projects). Each mini-project has analysis, design, coding, testing and integration. You finish the iteration with a demo to the user, and then do system tests. The development team gets use to delivering finished code for each mini-project. Don't wait till the end.

Transition

Beta testing, performance tuning and user training such as user guides.

2.3 Use Cases

A use case is a specific way of using the system by using some part of the functionality. Each use case constitutes a complete course of events initiated by an actor and it specifies the interaction that takes place between an actor and the system. A use case is thus a special sequence of related transactions performed by an actor and the system in a dialogue. The collected use cases specify all the existing ways of using the system.

2.3.1 Use Case Analysis

Use case analysis is performed to identify portion of system performing specific task .In use case analysis use cases, actors interacting with those use cases, and boundaries are identified. A use case comprises a course of events begun by an actor, and it specifies the interaction between actor and the system. All the use cases specify existing ways to use the whole system .It is interaction of actors with external or other system with system being designed in order to achieve a goal. Use case describes the functionality of the product to be constructed.

Actors

This is the entity that is outside the system boundary but interacts with the system. The actors I have identified, in my project are:

Normal User (Soft Keyboard User).

Soft keyboard User

A person who want to type text using soft-keyboard of the system and then translate it.

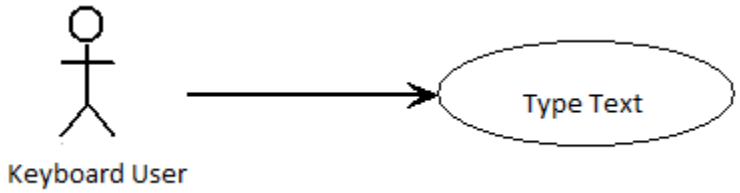
2.4 Use Case Diagram

A use case in a use case diagram is a visual representation of a distinct business functionality in a system. The key term here is "distinct business functionality." To choose a business process as a likely candidate for modeling as a use case, you need to ensure that the business process is discrete in nature.

2.4.1 Text Input:

Use Case: Text Input	
Use Case#	2.4.1
Actors:	Keyboard User
Purpose:	This use case is used to type text using soft-keyboard for translation.

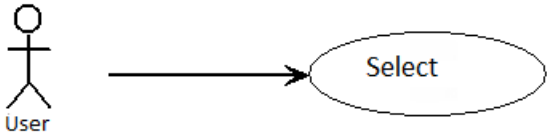
Use case diagram:



2.4.2 Select Use Case:

Use Case: Detail	
Use Case#	2.4.2
Actors:	User
Purpose:	This use case is used to show the search record

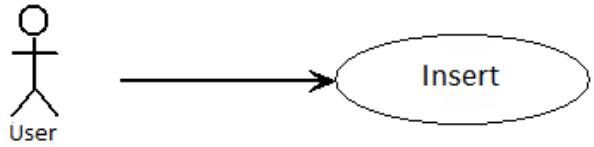
Use case diagram:



2.4.3 Add Detail:

Use Case: Detail	
Use Case#	2.4.3
Actors:	User
Purpose:	This is used to add record in fields

Use case diagram:

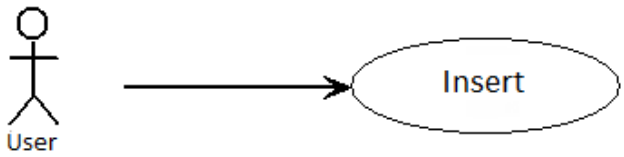


2.4.4 Update details:

Use Case: Text Input	
Use Case#	2
Actors:	User

Purpose:	This use case is used to update the record.

Use case diagram:



Chapter 3

SYSTEM DESIGN

3.1 System Design

The purpose of design is to create architecture for the evolving implementations. The design phase focuses on defining the software to implement the application. The design objective is to produce a model

of the system, which can be used later to build the system .The design goal, is to find the best possible design within the limitations imposed by the requirement and the physical and social environment in which the system will operate.

Object oriented design is a method of design encompassing the process of objects oriented decomposition and a notation for depicting logical and physical as well as static and dynamic models of system under design.

3.2 Activity Diagrams

It gives the pictorial representation for algorithm for function. Activity diagram is used to represent activities present in use cases. Basic need is that we want to make procedural design in UML. Operations in use cases in sequence are represented in activity diagram. Activity diagram are useful when we want to describe a behavior, which is parallel, or when we want to show how behaviors in several use cases interact.

3.3 Sequence Diagram

Sequence diagrams are used to show the flow of functionality through a use case .For one use case diagram there can be multiple sequence diagrams. For alternate course of actions there are separate sequence diagrams. Sequence diagrams are time dependent and tell which operation will be executed first. Sequence diagrams define a pattern of interaction among objects arranged in chronological order. These show the objects participating in interaction by the order of their life times and the messages being sent from one object to the other.

3.4 Detailed Sequence and Activity Diagrams

3.4.1 New Patient:

Preconditions:

1. User must click new Patient Button.
2. User must have the information required.
3. User must click save button to save the record

Flow of events:

Primary scenario:

1. Click on the “New Patient”.

2. User enter patient name.
3. User enter cnic number.
4. User enter date of birth
5. User selected male or female.
6. User enter blood group
7. User enter mobile number
8. User enter address

Exceptions:

1. Required fields are empty.
2. Record already saved.

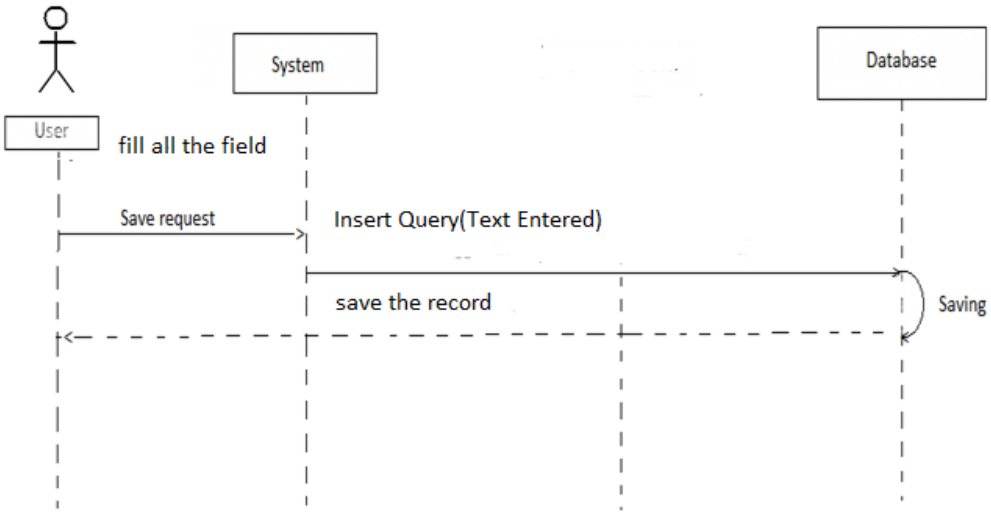
Secondary Scenario:

1. User can go back to main page at any time.

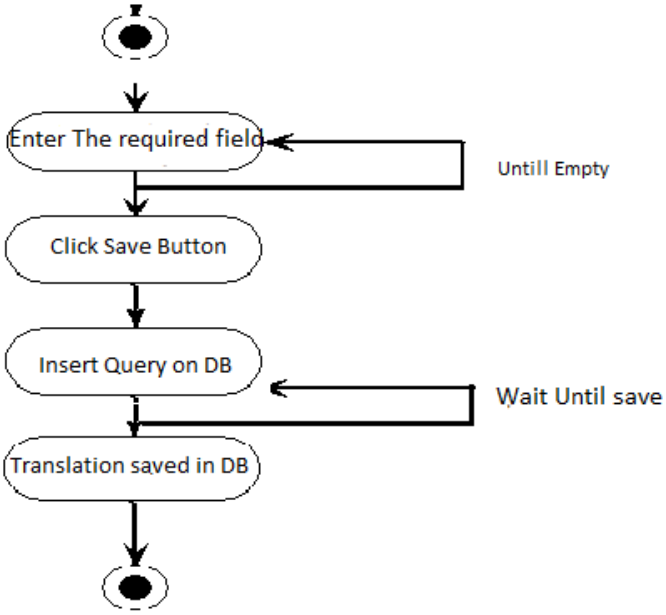
Post condition:

1. Saved the record for future use.

Sequence Diagram:



Activity Diagram:



3.4.2 All Patient:

Preconditions:

1. User must click new Patient Button.
2. User must have access to the list.
3. User must press one record for a while.

Flow of events:

Primary scenario:

1. Press one record at a time.
2. Press one record for a while.
3. User Edit the current record.
4. User View the current record.
5. User Delete the current record.
6. User Update the current record.
7. User Add new record.
8. User Click Current checkups.
9. User Checkup history.
10. User Click Current findings.
11. User Findings history.

Exceptions:

1. User Click none of them.

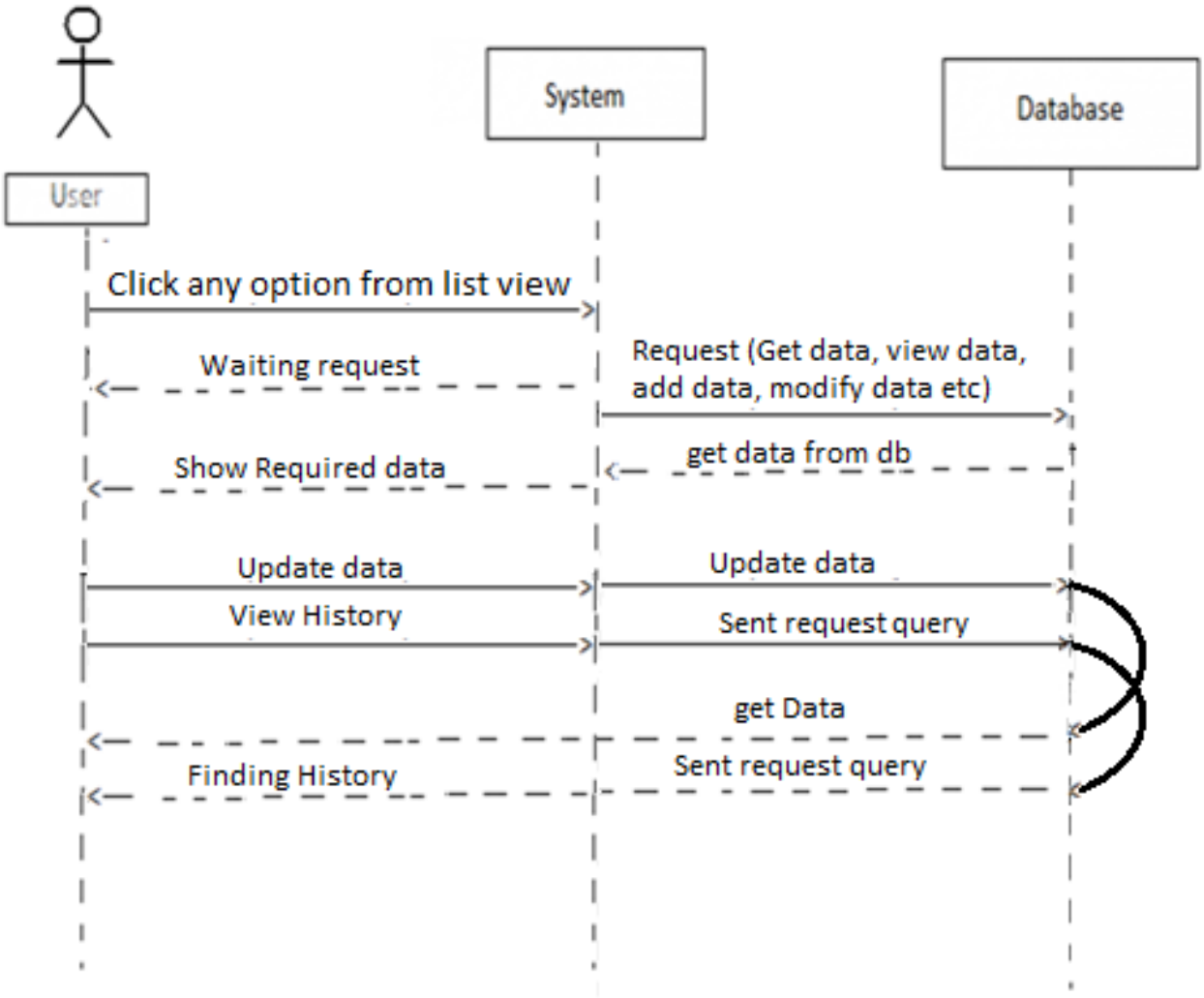
Secondary Scenario:

1. User can go back to main page at any time.

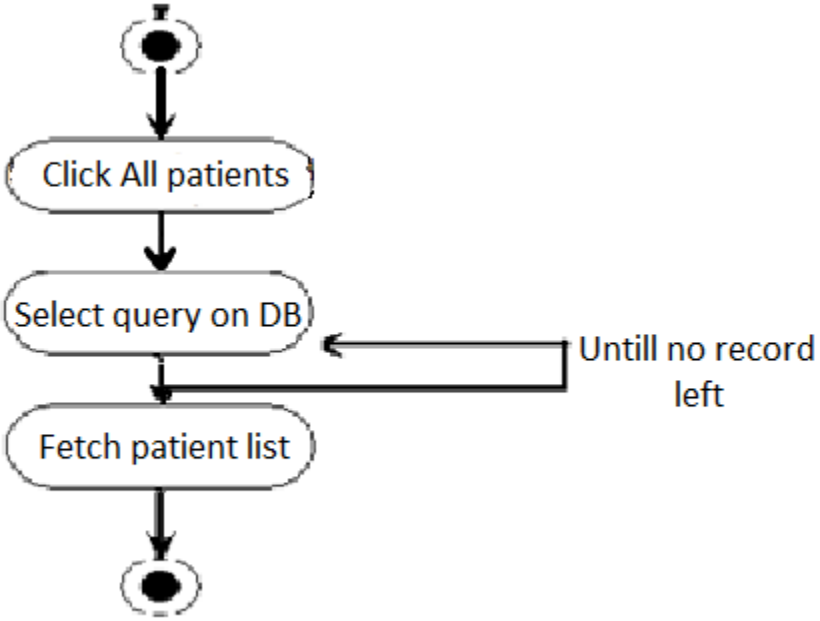
Post condition:

1. User must click the record for a moment.

Sequence Diagram:



Activity Diagram:



3.4.2 Save Record:

Preconditions:

1. User must click Save.
2. User have filled all fields already.

Flow of events:

Primary scenario:

1. Click on the “Save”.
2. Text entered will be saved.
3. Show “Saved” Message, if succeeded.
4. This use case ends here.

Exceptions:

1. Required fields are empty.
2. Record already saved.

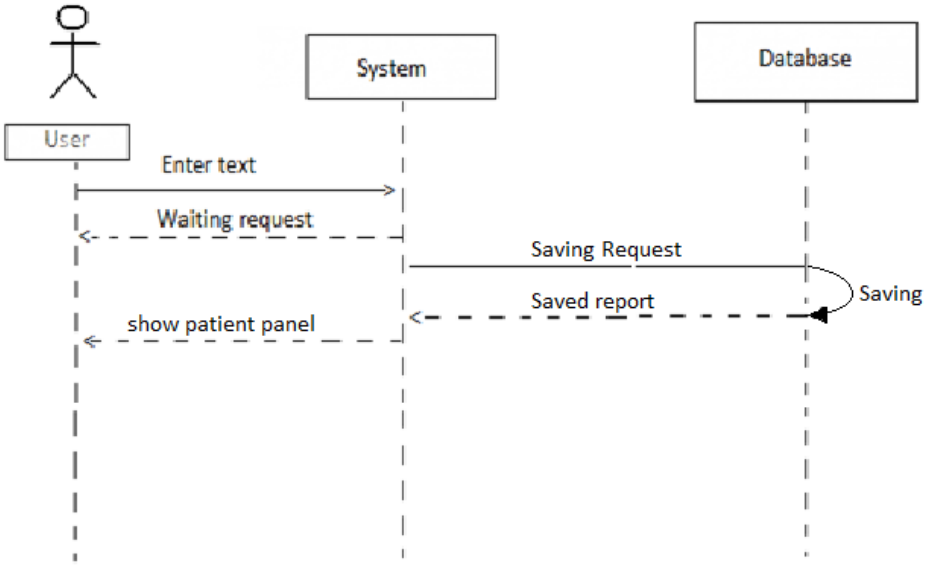
Secondary Scenario:

1. User can go back to main window page at any time.

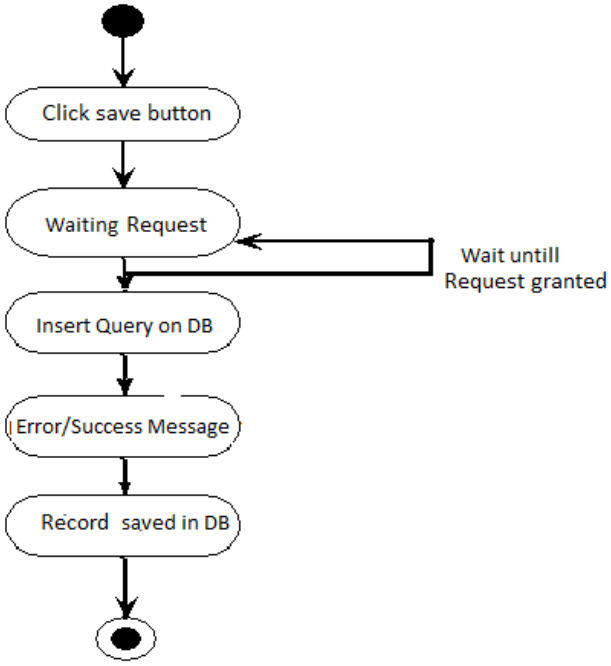
Post condition:

1. Saved translation for future use.

Sequence Diagram:



Activity Diagram:



3.4.4 Delete Record:

Preconditions:

1. Record must be Entered.
2. Entered Record must be saved and there must be at least one entry of it in database.

Flow of events:

Primary scenario:

1. Open All Patient menu.
2. All saved patients will be fetched from database and show in list.
3. Click Patient to be deleted for a while.
4. A list will be appeared of the actions that could be performed on.
5. Click “Delete” button to delete the record.
6. Toast message “Deleted” will be appeared if saved.
7. If not deleted toast message show “Not deleted option”.
8. This use case ends here.

Exception:

1. There must be some record in database.

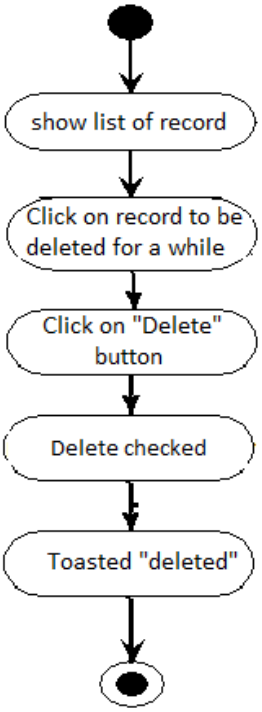
Secondary Scenario:

1. User can go back to main window any time.

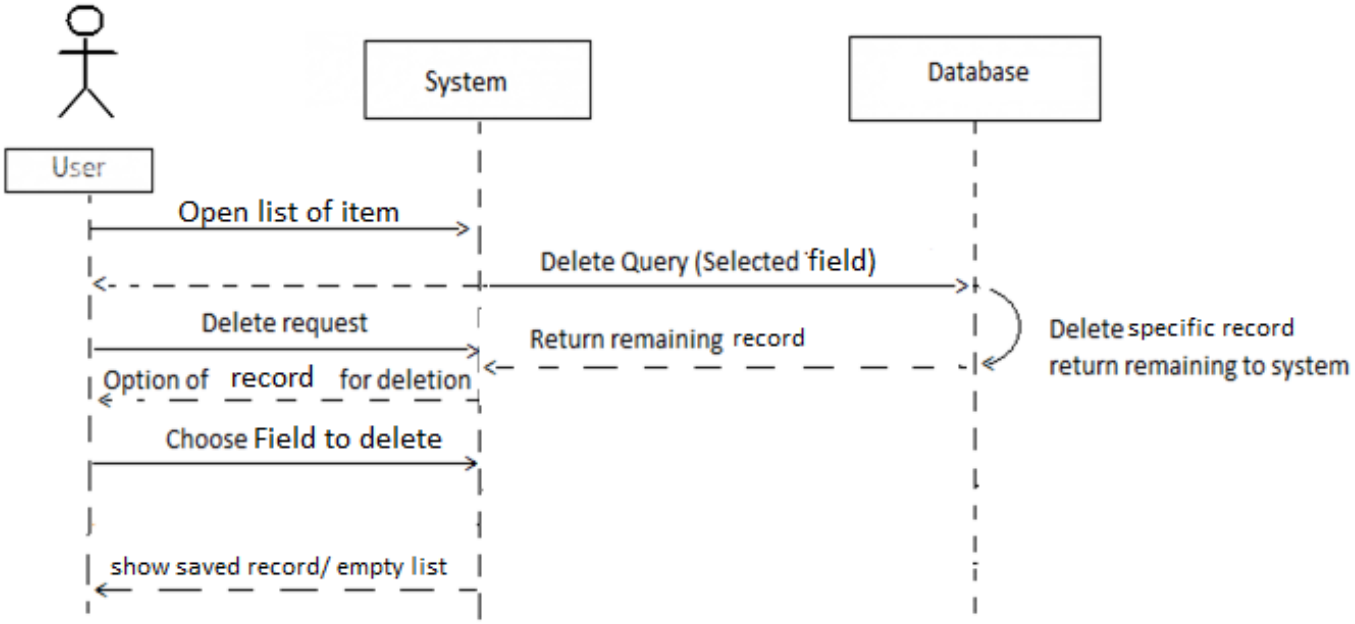
Post condition:

1. User must click for a while.
2. User must select some data to delete.

Activity Diagram:



Sequence Diagram:



Chapter 4

DATABASE DESIGN

4.1 Introduction to Database:

Database is a collection of information in a structured way. We can say that it's a collection of a group of facts e.g. Address book etc.

A Database consists of an organized collection of data for one or more uses, typically in digital form.

4.1.1 What is Database?

A Database is a collection of information organized in to interrelated tables of data and specification of data objects. A shared collection of logically related data organized in a way that facilitates data operations.

So, Database means the meaningful storage of data.

Feature of Database may include:

- Representing the information about a real world enterprise or part of enterprise.
- Collected and maintained to serve specific data management need of enterprise.
- Activities of the enterprise are supported by database and continuously update the database.

4.2 Database system:

It's essentially a computerized record keeping system. The Database itself can be regarded there are four basic components of database system.

4.2.1 Data Base Management System:

An application that's used to create, maintain and provide control access to Database.

- An application which allows you to create, store, organize, and retrieve data from a database.
- Example: MS Access, Oracle, SQLite, MySQL etc.

4.2.2 DBMS approach:

DBMS:

- Separation of data and metadata.
- Flexibility of changing metadata.
- Program-data independence.

Data access language:

- Standardize-SQL

- Easy Ad-hoc query formulation

System development:

- Less effort required
- Concentration on logical level design is enough
- Components to organize data storage
- Process queries, manage concurrent access, recovery from failures, manage access control are all available.

4.2.3 Three level architecture:

4.2.4 Data

Data is facts and figures that has been organized and categorized for a pre-determined purpose.

4.2.5 Hardware:

It's the machine which we use to store, access, manipulate and manages the data. It include secondary storage, processor, memory that is used to execution of database software.

4.2.6 Software:

All the requests from users for access to the database are handled by DBMS.

4.2.7 User:

The **Data Administrator** (DA) is responsible for the management of the data resource including database planning, development and maintenance of standards, policies and procedures, and conceptual/logical database design.

The **Database Administrator** (DBA) is responsible for the physical realization of the database, including physical database design and implementation, security and integrity control, maintenance of the operational system, and ensuring satisfactory performance of the applications for users.

The **logical database designer** is concerned with identifying the data (that is, the entities and attributes), the relationships between the data, and the constraints on the data that is to be stored in the database.

Once the database has been implemented, the application programs that provide the required functionality for the end-users must be implemented. This is the responsibility of the **application developers**

Naive users are typically unaware of the DBMS. They access the database through specially written application programs that attempt to make the operations as simple as possible.

Sophisticated users. At the other end of the spectrum, the sophisticated end-user is familiar with the structure of the database and the facilities offered by the DBMS.

4.3 Advantages of Database:

1. Program-Data independence
2. Data security
3. Data redundancy
4. Data consistency
5. Data sharing
6. Standards
7. Data quality
8. Program maintenance
9. Data accessibility

4.4 Normalization:

The process of decomposing the non-structured and anomalies container relation in to anomalies-free well-structured relations. Its purpose is to remove anomalies, that'll be produced from different operations and produce well-structured relations.

Well-Structured relation:

A relation is well-structured, if it contain minimum redundancy and no insertion, modification, and deletion anomalies. Anomaly is the error that occurs, when user perform some operation on the database.

4.5 Database design:

Database design is the core of system development. By knowing the requirements of the user, developer designs the database. Database design is technical discipline that is applied once the information domain of the database design has been defined. Therefore, the role of the system developer is to define the information to be contained in the database, the type of queries to be

submitted for processing the manner in which data will be accessed and the capability of the database.

Database design is a set of different activities. The database for the under discuss system is the result of different intermediate activities.

My database name is

MyDb

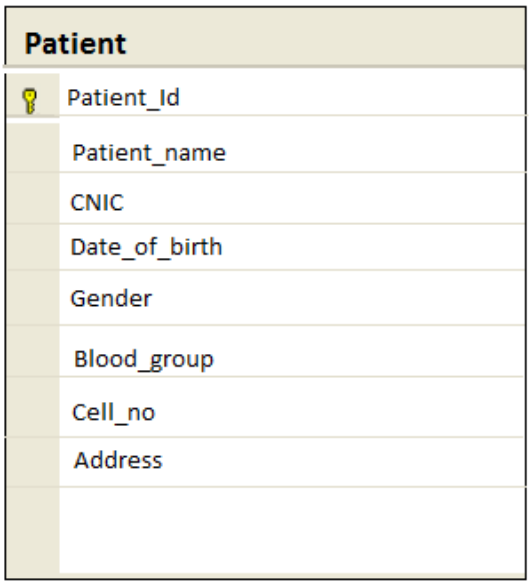
There are seven tables in my database:

1) **Patient**
2) **Company**
3) **My_Medicine**
4) **Patientcheckup**
5) **CheckUp_Details**
6) **My_Findings**
7) **Mine_Checkup**

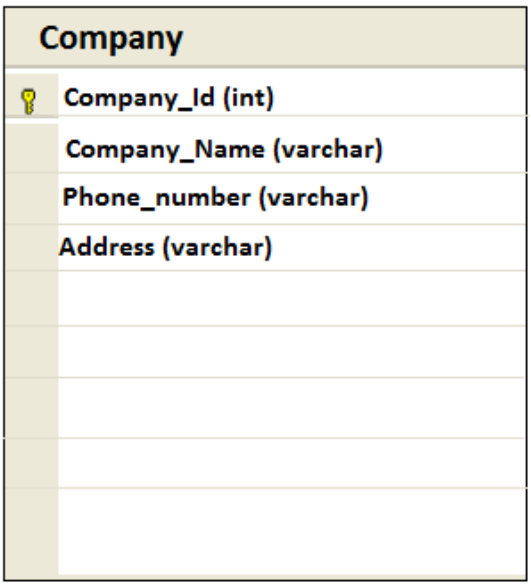
4.5.1 Design View:

The design view in sqlite3 of my databases:

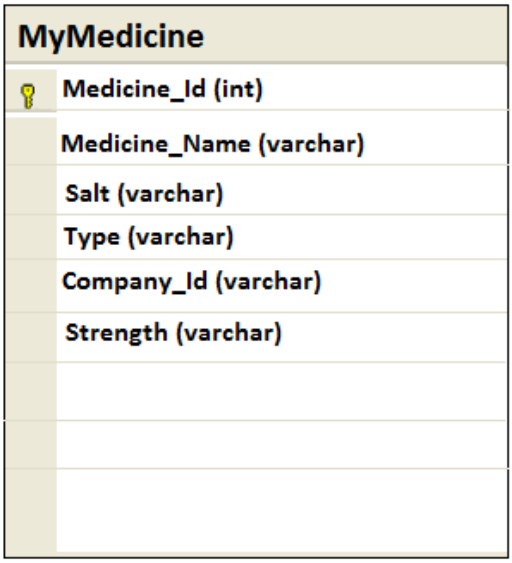
Patient Table:



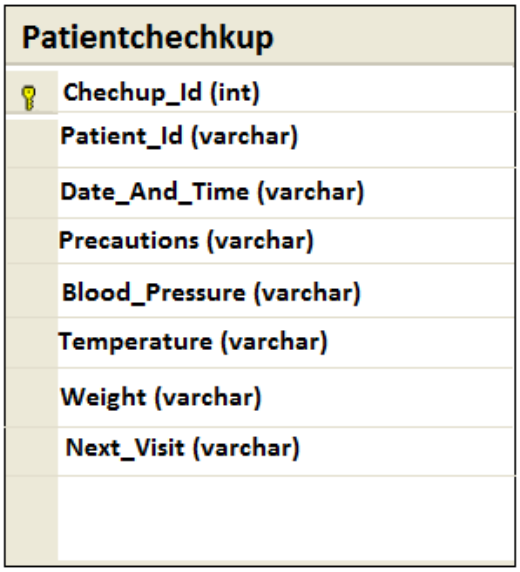
Company Table:



My_Medicine:

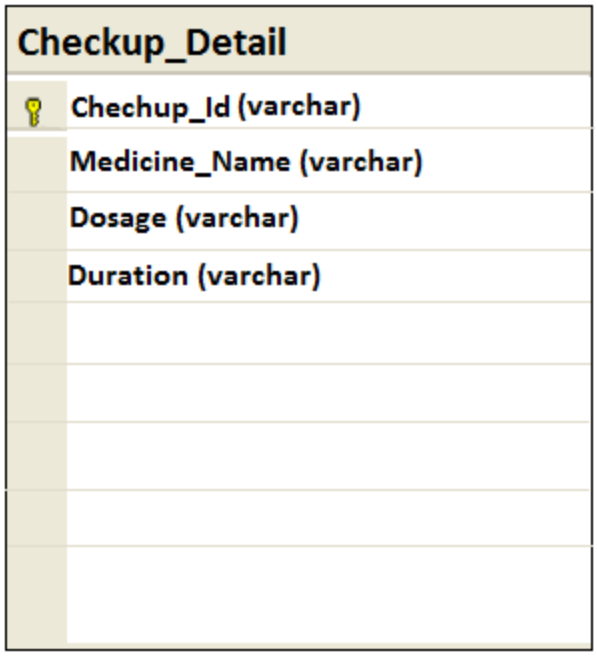


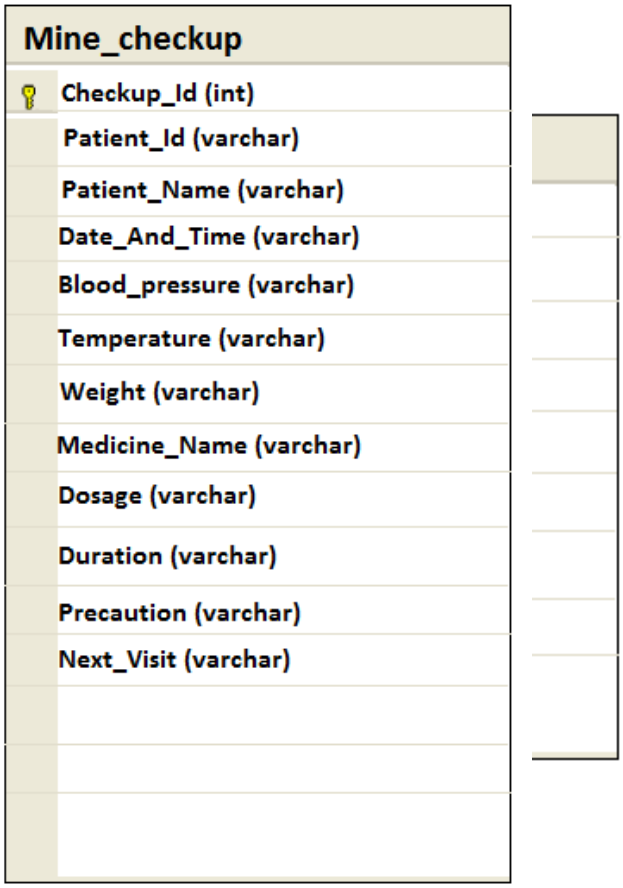
Patientcheckup:



Checkup_Deatil:

My_Findings:





Mine_checkup:

4.6 Data Modeling:

Data modeling in Software Engineering is the process of creating a data model by applying formal data model descriptions using data modeling techniques. The data requirements are recorded as conceptual data model with associated data definitions. Actual implementation of the conceptual data model is called logical data model.

After creating conceptual database design, we can represent that by using any modeling technique.

- UML
- ERD
- Relation Model
- Relational algebra

Chapter 5

IMPLEMENTATION

5.1 Introduction:

The “Patient in Pocket” is general purpose application that facilitates the user of Android Operating System to communicate with people effectively. This application contains rich interfaces for the users to interact with it. The choice of programming language is considering the system requirements and origination.

Language Selection:

5.1.1 Java

Java is a computer programming language that is concurrent, class-based, object-oriented, and specifically designed to have as few implementation dependencies as possible. It is intended to let application developers "write once, run anywhere" (WORA), meaning that code that runs on one platform does not need to be recompiled to run on another. Java applications are typically compiled to byte-code (class file) that can run on

any Java virtual machine (JVM) regardless of computer architecture. Java is platform-independent language. It have libraries for development for many platforms.

5.1.2 Java Language for Android platform:

Java Platform, Micro Edition, or **Java ME**, is a Java platform designed for embedded systems (mobile devices are one kind of such systems). Target devices range from industrial controls to mobile phones (especially feature phones) and set-top boxes. Java ME was formerly known as **Java 2 Platform, Micro Edition** (**J2ME**).

Java ME was designed by Sun Microsystems, acquired by Oracle Corporation in 2010; the platform replaced a similar technology, Personal. Originally developed under the Java Community Process as JSR 68, the different flavors of Java ME have evolved in separate JSRs. Sun provides a reference implementation of the specification, but has tended not to provide free binary implementations of its Java ME runtime environment for mobile devices, rather relying on third parties to provide their own.

As of 22 December 2006, the Java ME source code is licensed under the GNU General Public License, and is released under the project name phone ME.

What is J2ME or Java ME?

Java Platform, Micro Edition (Java ME) provides a robust, flexible environment for applications running on mobile and embedded devices: mobile phones, set-top boxes, Blu-ray Disc players, digital media devices, M2M modules, printers and more.

Java ME technology was originally created in order to deal with the constraints associated with building applications for small devices. For this purpose Oracle defined the basics for Java ME technology to fit such a limited environment and make it possible to create Java applications running on small devices with limited memory, display and power capacity.

5.1.3 Reason why I use J2ME:

Here are the reasons to use Java:

The Java Community

The best thing about Java is the huge community around it and that's why it's number one. I personally believe that without the great community Java wouldn't have become what it is today. Well there are many reasons that make it so great community and I'll mention the most important reasons.

For once they are a great resource, whether it comes to getting help or finding code examples regarding Java development. You have a problem with your code, no problems as there are literally hundreds of sites with forums and mailing lists where you can ask for some help to figure out what's wrong. You can ask for advice if you hit a wall while coding and don't know what solution would be best.

There are a great amount of sites with tutorials and code examples on how you code certain Java and android-based applications, where you can see how you go about in making a forum or whatever you might be wondering, or simply get inspiration to your projects. This point alone has a lot to do with Java and the community around it so much!

Learning Java is easy!

Java is because it's very easy to learn and get a hang of compared to some other languages. Learning Java isn't rocket science and with the great amount of help found on the internet it makes the learning process much easier.

Another thing that makes Java easy to learn for other programmers, especially the Perl, C and C-like language programmers (C and C++) is that the Java syntax is based on primarily C and Perl. This means that if you have experience with any of those languages you will pretty much be producing productive code instantly.

Performance

The performance of Java is great, it's very efficient. It's platform-independent, so we can execute one code on many platforms i.e. Code Once Use Everywhere policy.

The low cost

The price of Java is the amazing amount of zero. It's free and you can download the latest version of Java and its libraries at any given time. It's development platforms is free to purchase and need to cost for apps deployment.

Open Source, You can modify it

Open Source which means that you have complete access to its source code. If you want to modify or simply add something to the language, you can do it. Unlike commercial closed-source applications you don't have to wait for the manufacturer to release patches, if you have the knowledge you can do it yourself.

The built-In libraries

Java was designed to be used on the Web, Mobile development etc., because of this it has a lot of built in function, to be used for many different web related tasks. You can connect to web services and other network services, parse XML, send email, generate PDF documents, send emails and generate GIF images on the spot. This is just some of the functions you find in Java.

A part from the official libraries, some programmers have developed more libraries that can be found, whether it's a library with functions to work with audio or video or something completely different it most likely already exists. And that's another good part of Java; you can add custom libraries to it all after you needs.

Portability

Java is available for many different operating systems whether it's a free UNIX like operating system such as Linux or a system you paid for like Microsoft Windows. Well written code will usually work without modifications on different operating systems with no hassle. Of course there are functions in Java that only work on all operating system, so it not working on a different wouldn't be any surprise.

Strong Object-oriented support

Java has very well designed object-oriented features. Its pure object oriented programing language. It now has features like inheritance, private and protected methods and attributes,

abstract classes and methods, constructers, interfaces and destructors. There are even features of built in iteration behavior which was available.

If you program in C++ or C then all this will be familiar to you, and Java's syntax being very similar as well will just make it all a lot easier.

Has interfaces to large variety of database systems

Whether you're using MYSQL, PostgreSQL, MYSQL, Oracle, dbm, FilePro, Hyper Wave, Indormix, InterBase, Sybase databases, you can connect to those with PHP, there are many more but I found that list long enough.

Android OS also has a built in SQL interface to a flat file called SQLite for Java libraries.

Support available

Also for the non-commercial users there's support available in another form, the Java community which I mentioned earlier. The following are the technologies that are used to handle these tasks efficiently.

Android Market:

There is easy to accessible market for buyers and seller at international level. Google is its managing organization. The market is at google-play website, where developer can easily publish their apps easily and with the lowest cost. This market have customers from all over the world. So it's good place for marketing of software. It also facilitates the customer. It gave access of its customer to the market with cost equal zero comparatively.

5.2 Software Selection:

The no. of tools we will use in our project may increase during system development. However estimation is given below:

5.2.1 Software Technologies:

1. Java (J2ME)

2. Eclipse/ Android Studio
3. Web services
4. Sqlite3
5. Browser (e.g. Firefox)
6. GenyMotion Emulator
7. Adobe Photoshop
8. Microsoft Word

5.2.2 Hardware Technologies:

1. Pentium 4 1.8GHz or up
2. 256 MB Ram or up
3. Android Smart Phone: Samsung A300
4. Printer: HP Inkjet 810

Unified Modeling Languages (UML):

5.2.3 Background:

The Unified Modeling Language (UML) can be used for specifying, visualizing and constructing the elements of software systems. It can also be used for business modeling. The language ensures that large and complex systems can be modeled successfully. Due to the rapid increase in the complexity of system requirements, successful modeling is an essential task. UML is a proposal for a single common modeling language for object oriented system. As such it may become one of the most significant developments in object oriented software development. Recent developments include the adaptation of the UML specification by the Object Management Group.

5.2.3 Objectives:

1. Undertake use case analysis as part of the analysis and specification process for computer systems
2. Use different levels of object models during the specification and design of computer Systems.
3. Develop and refine sequence diagrams that guarantee the system specified can be implemented using your object model.
4. Develop State Models to assist in the specification of systems.
5. Use design patterns to assist your designs.
6. Use packages to structure large systems.

7. Understand the elements of technical domain design and be aware of many of the additional issues to be considered during technical domain design (e.g. persistence and database interfacing, distribution, user interface, external system interface, etc.)
8. Understand how the techniques of UML can be used to cross check each other, minimizing the potential effect on development projects.

5.2.4 Techniques:

The most commonly used techniques are: Use case, Sequence Diagram and activity diagram. Domain Experts can understand these diagrams and provide valuable feedback.

Activity Diagram:

1. Focuses on work flow processes.
2. Can show many objects over many use cases (implementation of methods).Shows the steps people go through in doing their jobs.
3. Encourages finding parallel processes and eliminating unnecessary sequences in business process.

Iterative Development:

The process of developing and releasing software in pieces, not in one big bang is known as iterative.

Patterns:

1. Look at the results of the process.
2. Describe common ways of doing things.
3. Look for repeating themes in designs.

Use Case:

1. Drives the whole development process.
2. Describes the typical interaction that is user has with the system in order to achieve some goal.

3. Each ‘Use case’ should indicate a function that the user can understand and that has value for that user.
4. Proves the basis of communication between the sponsors and developers in planning the project.
5. Construction planning is built around delivering some use cases in each iteration.
6. Basis for system testing.

Chapter 6

SYSTEM TESTING

6.1 Verification and Validation

6.1.1 Verification

Checks if the product under construction meets the requirements definition.

6.1.2 Validation

Checks if the product's functions are what the customer really wants. Within the area of Validation we could examine techniques for any one of the phases of the development lifecycle. In this phase two complementary processes are predominant - Testing and Debugging.

6.2 Testing

It's the process of identify the correctness, completeness, security, and quality of developed software. Testing is the process of technical investigation, performed on behalf of stockholders, that's intended to reveal quality related information about the product with respect to the context in which it's intended to operate.

Exercising the program using data similar to the real data that the program is designed to execute, observing the outputs, and inferring the existence of errors or inadequacies from anomalies in that output.

6.2.1 Debugging

The location and correction of errors found during testing.

6.2.2 Overview of Testing

Software robustness is a problem that everybody cares about but few people address in their products. The average project has several weeks devoted to testing, mostly in the weeks before deployment. Of course, most software ends up behind schedule and over budget, and testing is the first thing to get reduced or cut.

Testing is the most common way to increase confidence in the correctness and reliability of software. Testing is an important phase during software development life cycle, and shows the stability of the product. Studies report that testing consumes about half of the cost of software development.

Software testing is the process of exercising a computer program with predetermined inputs and comparing the actual outputs with the expected outputs.

Software testing is any activity aimed at evaluating an attribute or capability of a program or system and determining that it meets its required results. Testing can uncover failures, show whether specifications are met for specific test cases, be an indication of overall reliability, and can be used for large systems. However, testing cannot show the absence of errors or prove that a program will behave as expected.

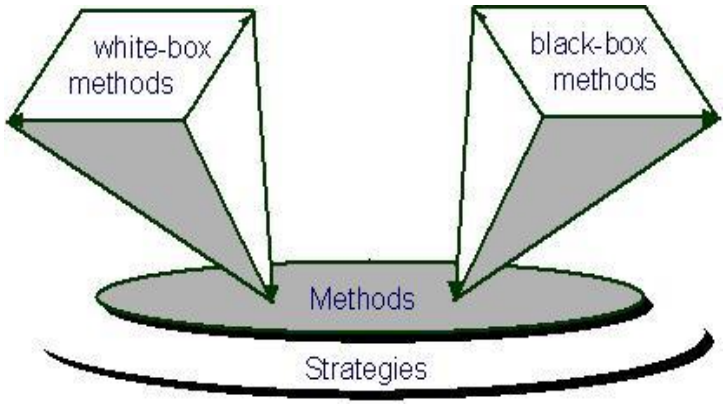
6.2.3 Objective of Testing

The purpose of testing can be quality assurance, verification and validation, or reliability estimation. The overall objective of the testing is to find the maximum number of errors in minimum amount of effort. Testing can find defects whose

consequences are obvious but which are buried in complex code, and thus will be hard to detect when inspecting.

6.3 Testing Strategies

We have to test our applications on different levels during development, as listed below:



6.3.1 White Box Testing

Term white box testing has been used for testing methodologies dealing with the source code. The idea of white box tests is to test all code lines to make sure that they work properly. White Box Testing implies access to the code of the application and the ability to test every line of code within it. Use of white-box testing is to test interfaces for robustness. Analysis of the code may indicate possible weaknesses related to violations of the spec, guiding the selection of test data to exercise specific cases that might cause an undesirable failure state.

White box testing is concerned only with testing the software product; it cannot guarantee that the complete specification has been implemented. White box testing is much more expensive than black box testing. It requires the source code to be produced before the tests can be planned and is much more laborious in the determination of suitable input data and the determination if the software is or is not correct.

6.3.1.1 Class Level Testing

Every class of the business logic that will be part of a component has to be tested.

6.3.1.2 Component Level Testing

For every component we create a counter component that looks like a mirror to the original component. This counter component has a receptacle for

every facet of the original component and vice versa. Connecting a component with its mirror component with a simple test client allows component developers to quickly isolate component level errors in the business code. The software being tested as intended for integration with other pieces rather than as a complete system in itself.

6.3.1.3 Assembly Level Testing

After testing each component, we have to be sure that a set of connected components, the component assembly, also works correctly.

6.3.2 Black box testing

Black box testing implies that the selection of test data as well as the interpretation of test results is performed on the basis of the functional properties of a piece of software. Black Box Testing is testing without knowledge of the internal workings of the item being tested. For example, when black box testing is applied to software engineering, the tester would only know the "legal" inputs and what the expected outputs should be, but not how the program actually arrives at those outputs. It is because of this that black box testing can be considered testing with respect to the specifications; no other knowledge of the program is necessary. For this reason, the tester and the programmer can be independent of one another, avoiding programmer bias toward his own work.

6.4 Test Cases:

6.4.1 Test case: 1

System:

Patient in Pocket

Test1:

Enter the main menu, select patient.

Test2:

Enter patient record.

Test3:

Save the record.

Instruction1:

1. Press patient.
2. Enter all text fields.
3. Press save button.

Instruction2:

1. Select all patients to view the list.

Instruction3:

1. Click specific record for a while.
2. Explore other options.

Expected result:

1. Record entered/ saved.
2. View record list.
3. View sub options of the record.

Actual result:

Messsage shows “No record to show”.

Chapter 7

FUTURE ENHANCEMENT

7.1 Future Enhancements:

The Patient in Pocket System is the most complete software package available for working offline. It handle all the basic working/daily working of the doctors and physicians. It not only comprise the record of patient but a complete clinic record in it. Record of the pharmacy medicines and record of the companies and organization associated with.

But still it is limited software and a lot more features can be added to it to make it look more enhanced. The following are some enhancements I think can be done in the future to my translation system.

7.1.1. Online the System:

This application will work online, so that it will be accessible to everyone. Once the system get online it will enhance many features. Doctors and patients will communicate online with each other. A patient would never miss his appointment. And it will also be convenient for doctors to attend the meeting with his staff online.

7.1.2. Chat Box :

Create an application for Android OS, for online chat. There will be a online chat portal, where patient will directly communicate with doctor. Patient unable to travel long distances will find it easy to have a doctors, sitting at long distance, at home.

7.1.3. Incoming/outgoing Call:

Create an application for Android OS, for calls, where doctor can call to the meeting staff.

7.1.4. Mail Inbox/outbox:

Create an application for Android Operating System, all the reports, x-rays and CT scans will be directly delivered to the doctor via mails.

7.1.5. Online Payment System:

An application will be developed on Android OS, where patient will pay for the checkups online via debit/credit cards. It will be helpful for patient and doctor to pay/ receive the dues.

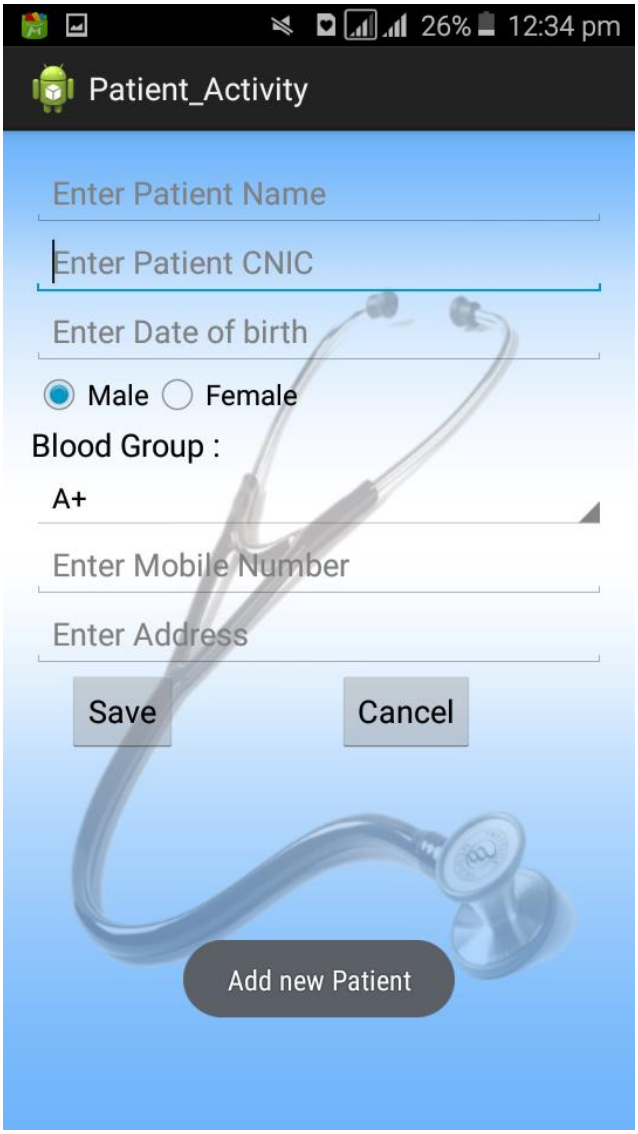
Chapter 8

USER MANUAL

Main Activity (GUI):

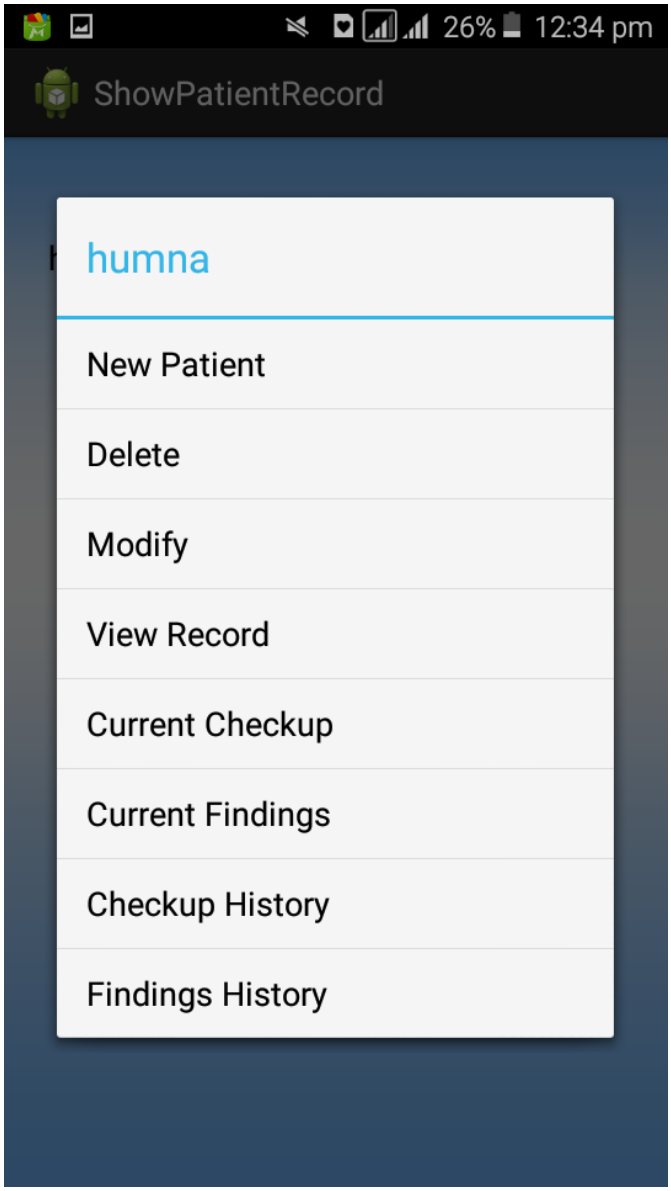
MyPatient Activity:

Patient Activity Portal:

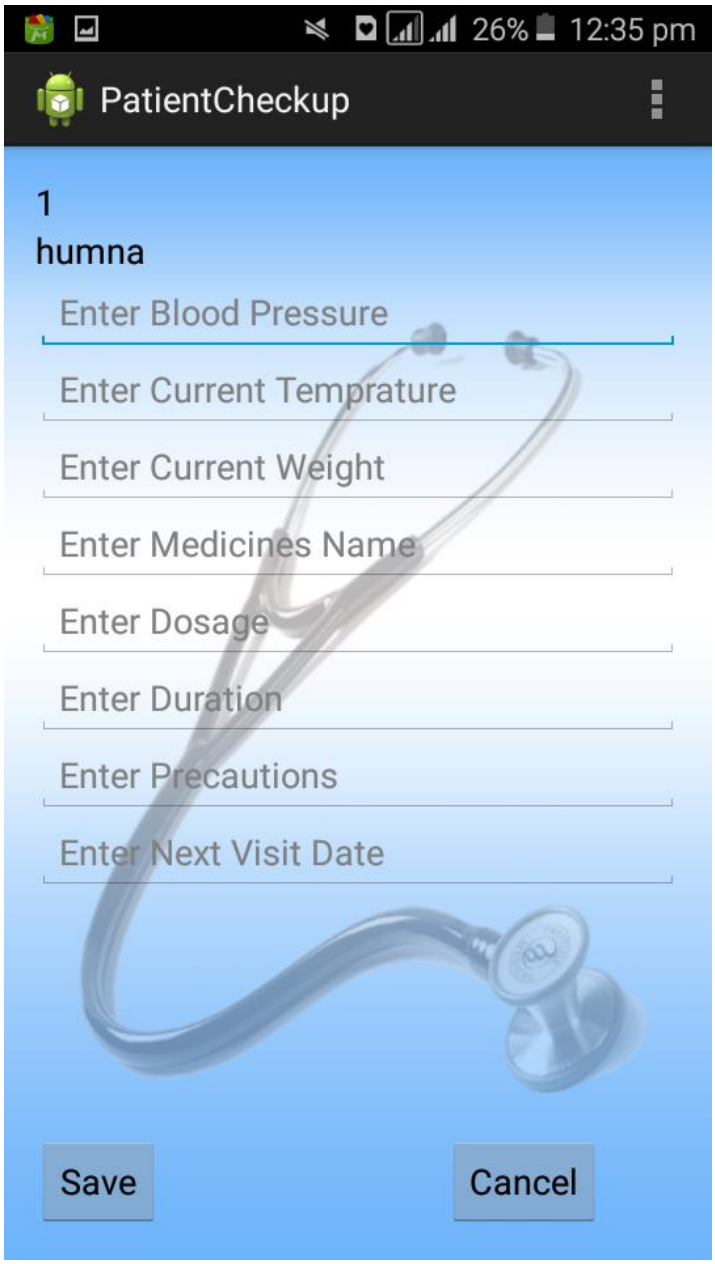


Show Patient Record:

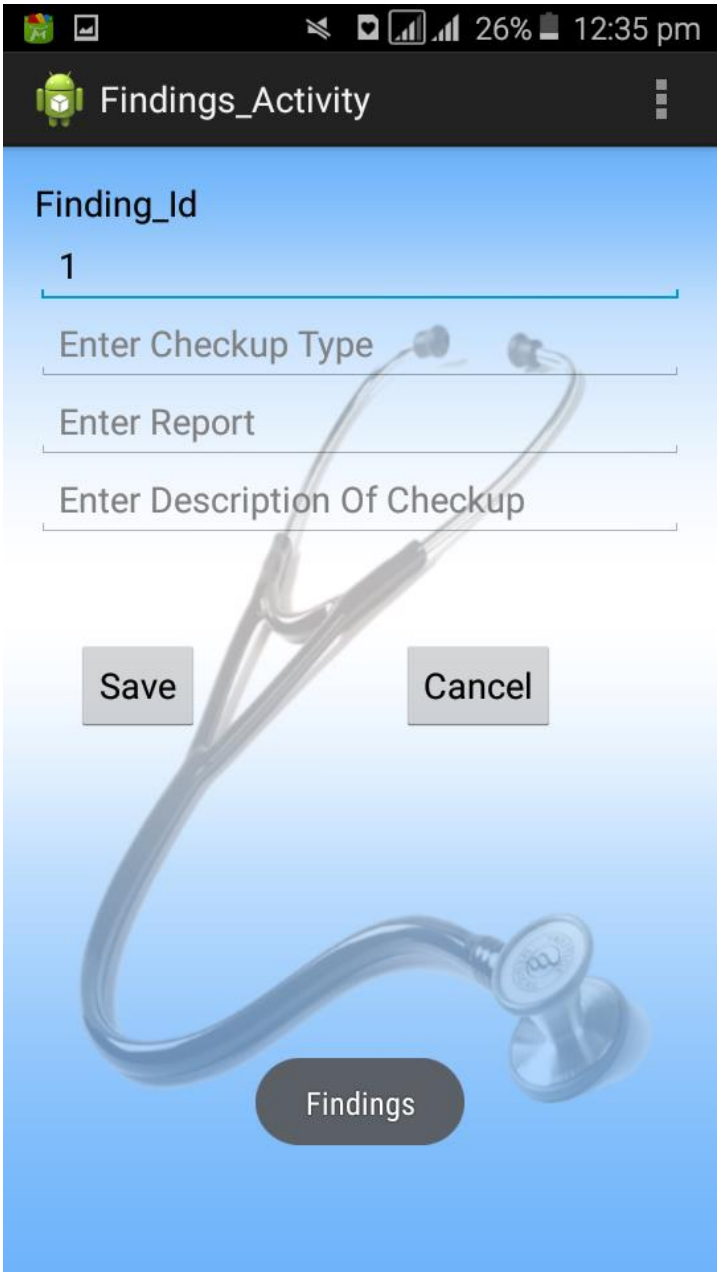
Menu list:



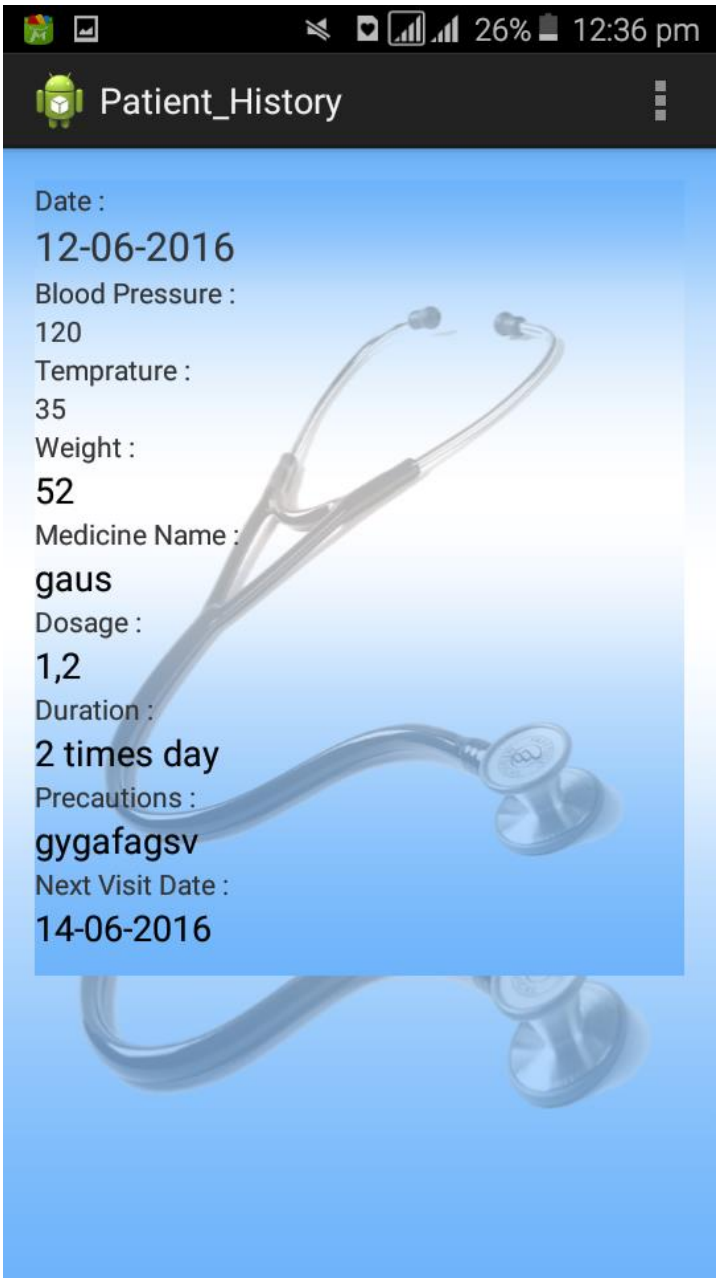
Patient Checkup Activity:



Finding Activity:



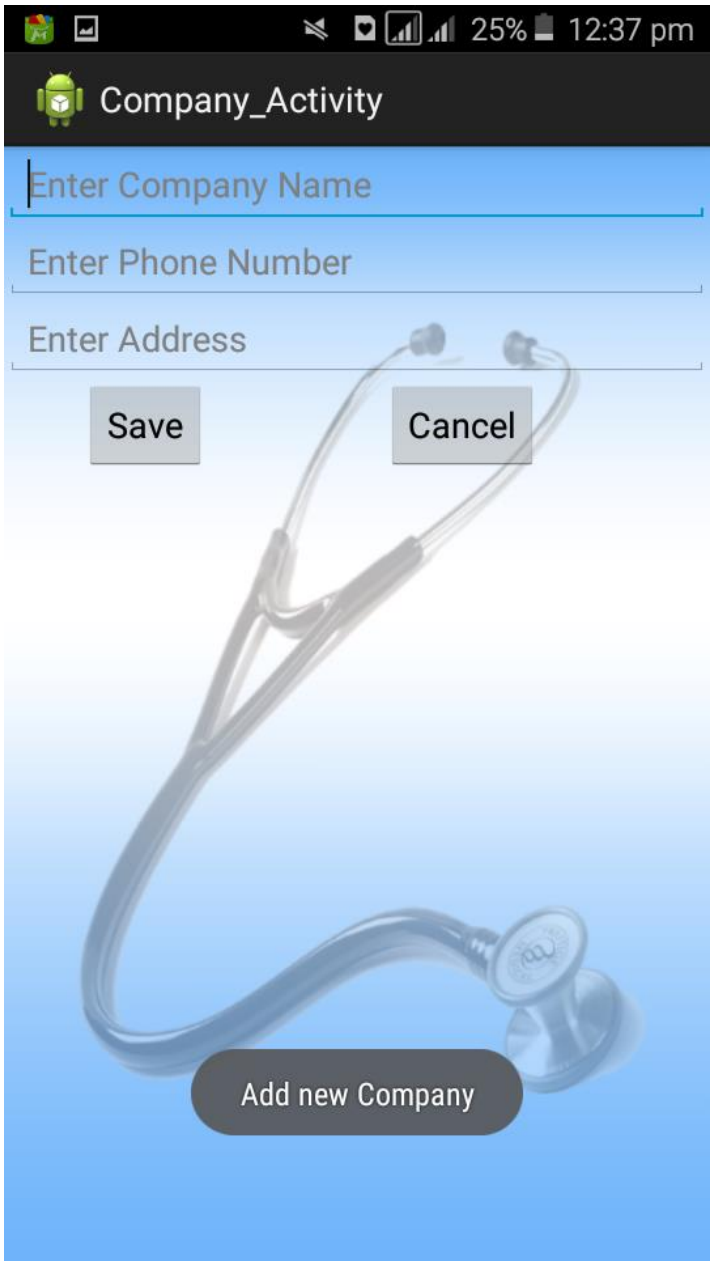
Patient Histroy Listview:



Finding History Listview:

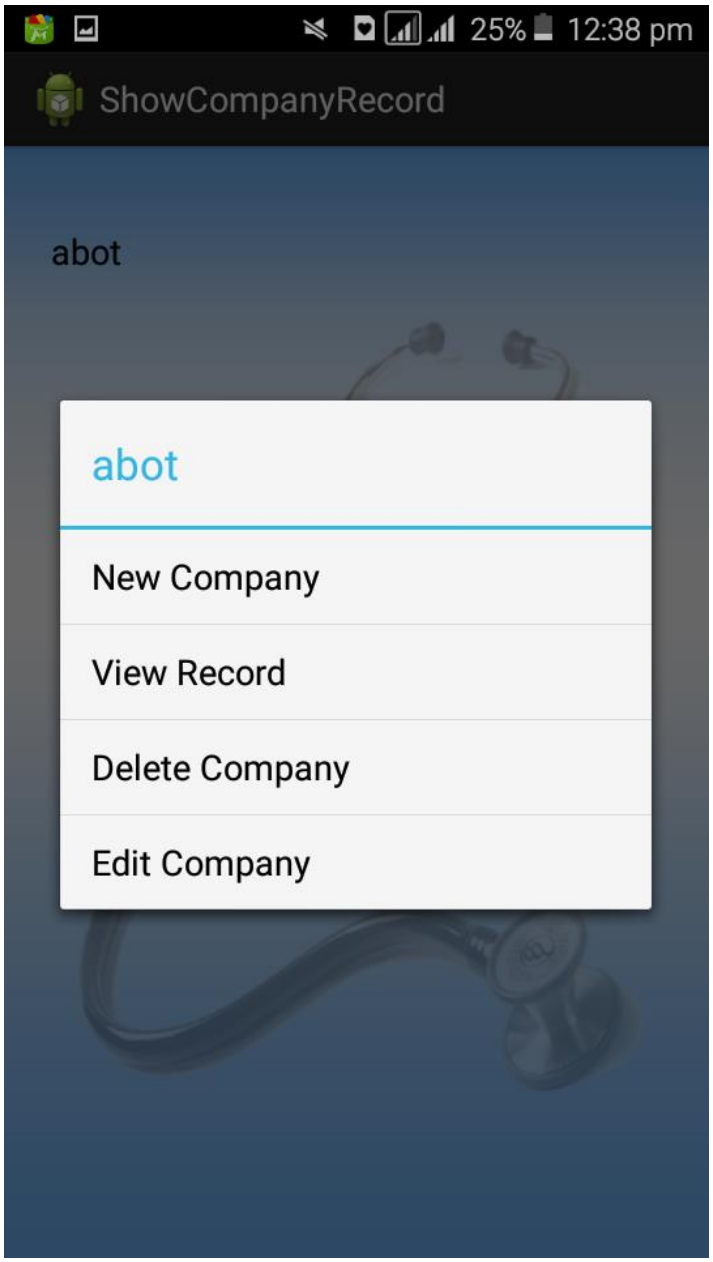
MyCompany:

Company Activity:



Show Company Record:

Menu List:



My Medicine:

Show Medicine Record:

Show Medicine Listview:

WEBSITE REFERENCES:

https://www.google.com

https://www.microsoft.com

https://w3school.com

https://www.stackoverflow.com

https://www.facebook.com

http://www.tutorialspoint.com/android/android_sqlite_database.htm

https://www.smartdraw.com/healthcare/examples/

APPENDIX:

SYSTEM REQUIREMENTS

Hardware requirements:

Developer's requirements

- Mini Processor Pentium III OR Higher
- Mini RAM 128 MB OR Higher
- Hard disk 10 GB OR Higher

Software requirements:

Developer's requirements

Operating system:

Win XP Pro/ Win NT/Win 2000 Pro /Win 7/ Win 8/Win 8.1

And Android Operating system

Development tools:

Eclipse/Android Studio, Emulator (bluestacks *Optional) /Android Smart Phone, sqlite.

Other software:

Microsoft Office 2013, Adobe Photoshop.